

第一部分总结

知识点：

1. DSP的总体架构——改进的哈佛结构
2. DSP的存储空间分配和应用——F28027.cmd
3. DSP的时钟配置——DSP2802x_SysCtrl.c
4. DSP的WatchDog——DSP2802x_SysCtrl.c
5. DSP的CPUTimer——DSP2802x_CpuTimers.c
6. DSP的GPIO与AIO——DSP2802x_Gpio.c
7. DSP的外部中断
8. DSP的中断系统
9. DSP的应用程序开发

DSP芯片的结构特征

- 为了适应快速数字信号处理运算的要求，DSP芯片普遍采用了**特殊的硬件和软件结构**，以提高其数字信号处理的运算速度
- DSP芯片的主要结构特征有:采用了**哈佛结构**、流水线技术、硬件乘法器和特殊DSP指令等

1.哈佛结构

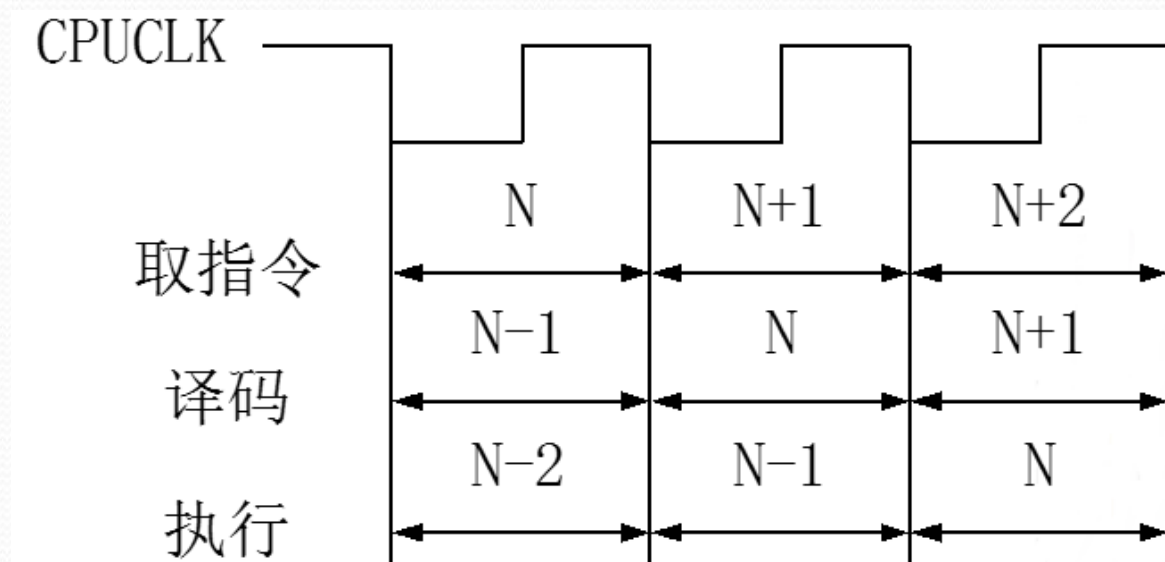
- 哈佛结构是一种并行体系结构，主要特点是将程序和数据存储在不同的存储器空间，对程序和数据独立编址，独立访问。
- 而且在DSP中设置了数据和程序两套总线，使得取指令和执行能完全重叠运行，提高数据吞吐量

哈佛结构—改进

- 为了进一步提高速度和灵活性，TMS320系列产品中，在哈佛结构上作了改进：
- 一是允许程序存储在高速缓存中，提高指令读取速度；
- 二是允许数据存放在程序存储器中，并被算术运算指令直接使用，增强芯片的灵活性。
- 另外DSP中的双口RAM(DARAM)及独立读写总线使数据存取速度提高

2.流水线技术

DSP芯片广泛采用流水线技术，增强了处理器的处理能力。TMS320系列流水线深度为2~6级不等，也就是说，处理器在一个时钟周期可并行处理2~6条指令，每条指令处于流水线的不同阶段。图1.2为三级流水线操作的例子。在三级流水线操作中，取指令、指令译码和执行可以独立地处理，这样DSP可以同时处理多条指令，只是每条指令处于不同处理阶段



3.硬件乘法器

在数字信号处理的许多算法中，需要做大量的乘法和加法。DSP芯片一般都有一个硬件乘法器，在TMS320系列中，一次乘累加最少可在一个时钟周期完成

4.特殊DSP指令

DSP芯片的另一个特点就是采用了特殊的寻址方式和指令。比如，TMS320系列的位反转寻址方式，LTD、MPY、RPTK等特殊指令。采用这些适合于数字信号处理的寻址方式和指令，进一步减少了数字信号处理的时间

DSP存储空间分配

	Data Space	Prog Space
0x00 0000	<i>M0 Vector RAM (Enabled if VMAP = 0)</i>	
0x00 0040	M0 SARAM (1K x 16, 0-Wait)	
0x00 0400	M1 SARAM (1K x 16, 0-Wait)	
0x00 0800	Peripheral Frame 0	Reserved
0x00 0D00	PIE Vector - RAM (256 x 16) (Enabled if VMAP = 1, ENPIE = 1)	
0x00 0E00	Peripheral Frame 0	
0x00 2000	Reserved	
0x00 6000	Peripheral Frame 1 (4K x 16, Protected)	Reserved
0x00 7000	Peripheral Frame 2 (4K x 16, Protected)	
0x00 8000	L0 SARAM (4K x 16) (0-Wait, Secure Zone + ECSL, Dual Mapped)	
0x00 9000	Reserved	
0x3D 7800	User OTP (1K x 16, Secure Zone + ECSL)	
0x3D 7C00	Reserved	
0x3D 7C80	Calibration Data	
0x3D 7CC0	Get_mode function	

DSP存储空间分配

0x3D 7CE0	Reserved
0x3D 7E80	Calibration Data
0x3D 7EB0	Reserved
0x3D 7FFF	PARTID
0x3D 8000	Reserved
0x3F 0000	FLASH (32K x 16, 4 Sectors, Secure Zone + ECSL)
0x3F 7FF8	128-Bit Password
0x3F 8000	L0 SARAM (4K x 16) (0-Wait, Secure Zone + ECSL, Dual Mapped)
0x3F 9000	Reserved
0x3F E000	Boot ROM (8K x 16, 0-Wait)
0x3F FFC0	Vector (32 Vectors, Enabled if VMAP = 1)

应用举例：

总体要求：做一个计时器，分辨率到0.01秒

具体：1) 显示XX.XX（小时与分钟）

XX.XX（秒与0.01秒）

2) 按键启停：启动—>停止—>清零—>启动

3) 跑马灯

分析：

1， 用到的外设：

CPUTimer0， 外部中断和GPIO

2， CPUTimer0完成计时功能

3， 外部中断完成控制功能

4， GPIO完成显示功能

5， 中断考虑： 定时器0中断和外部中断

应用程序举例:

硬件资源:

- 1) CPUTimer0
- 2) 外部中断GPIO12
- 3) LED数字显示:
AI02(Data), AI04(Clk), AI06(CS)
- 4) LED发光管:
GPIO0~GPIO3

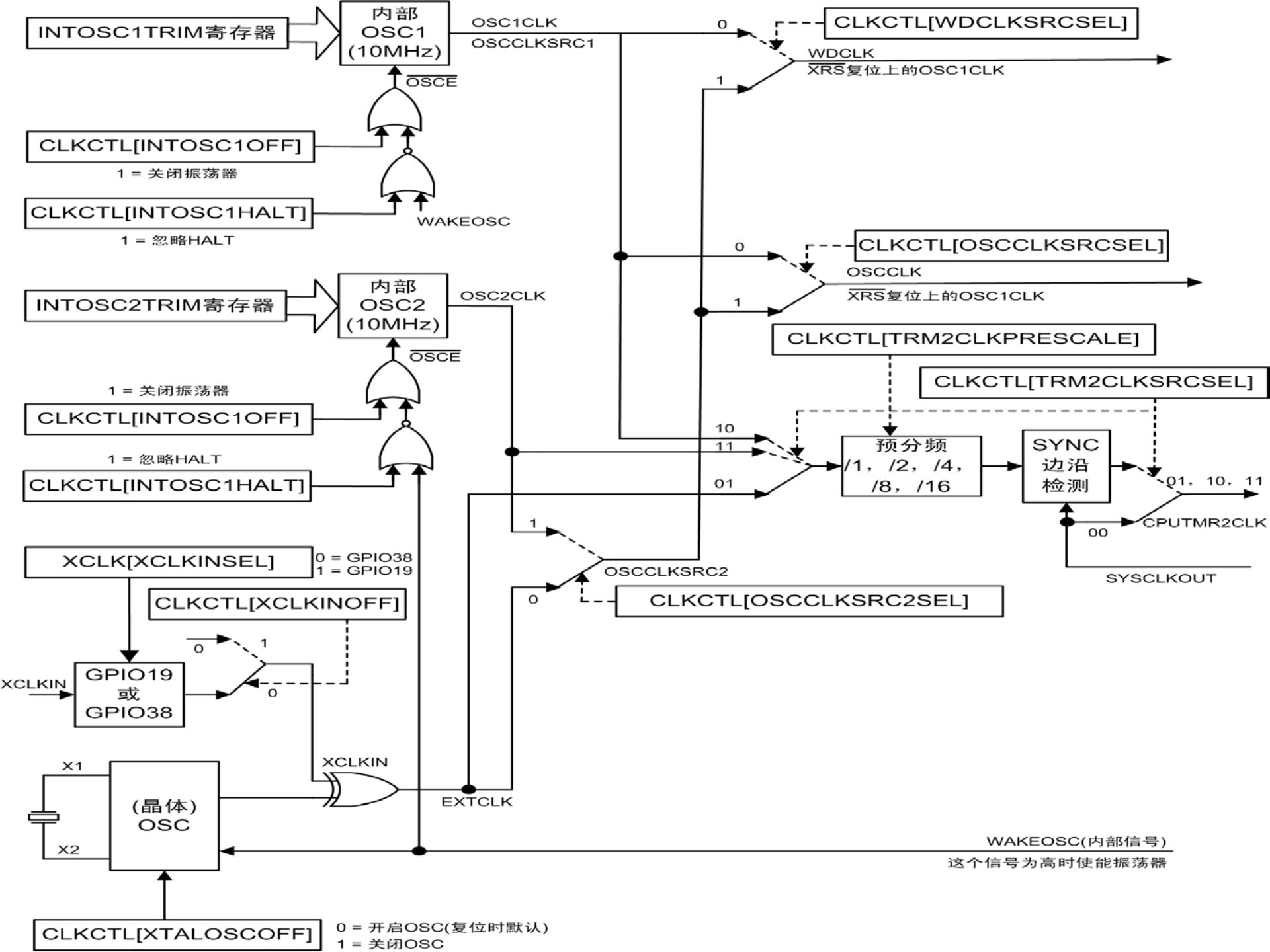
应用程序举例：

系统初始化：

- 1) WD初始化，关掉
- 2) 时钟源初始化，四选一
- 3) PLL初始化，12, 2, 60M
- 4) 外设时钟初始化
- 5) FLASH初始化，2, 2, 2

Watchdog 计数器操作方式

1. 0X55 & 0XAA 序列的顺序写入
2. WDDIS 位的置一 (WDOVERRIDE = 1)



配置输入时钟源 & XCLKOUT 寄存器 (XCLK)

XCLK寄存器用来选择XCLKIN输入的GPIO引脚和配置XCLKOUT引脚的频率。

7	6	5	2	1	0
保留	XCLKINSEL	保留		XCLKOUTDIV	
R-0	R/W-1	R-0		R/W-0	

图例：RW=读/写；R=只读；-n=复位后的值

4

位	域	值	描述 ⁽¹⁾
15-7	保留		保留。
6	XCLKINSEL	0 1	XCLKIN源选择位：这个位选择XCLKIN输入源。 GPIO38是XCLKIN输入源（这也是JTAG端口的TCK源）。 GPIO19是XCLKIN输入源。
5-2	保留		保留
0-0	XCLKOUTDIV	00 01 10 11	XCLKOUT分频比：这两个位选择XCLKOUT频率相对于SYSCLKOUT的比率。比率是： XCLKOUT = SYSCLKOUT/4 XCLKOUT = SYSCLKOUT/2 XCLKOUT = SYSCLKOUT XCLKOUT = Off

XCLK寄存器的XCLKINSEL位由复位引脚复位

配置器件时钟域 (CLKCTL)

CLKCTL寄存器用来选择时钟源，也用来配置时钟故障期间器件的行为

15	14	13	12	11	10	9	8
NMIRESETSEL	XTALOSCOFF	XCLKINOFF	WDHALTI	INTOSC2HALTI	INTOSC2OFF	INTOSC1HALTI	INTOSC1OFF
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
7	5	4	3	2	1	0	
TMR2CLKPRESCALE			TMR2CLKSRCSEL		WDCLKSRCSEL	OSCCLKSRC2SEL	OSCCLKSRCSEL
R/W-0			R/W-0		R/W-0	R/W-0	R/W-0

图例：RW=读/写；R=只读；-n=复位后的值

位	域	值	描述
15	NMIRESETSEL	0 1	NMI复位选择位：该位在检测到一个缺少时钟条件时直接产生 信号和使用 复位两者之间进行选择：没有任何延迟地直接驱动（复位时的默认状态）。NMI看门狗复位（ ）启动。 注：不管作何选择，都会产生 信号。

位	域	值	描述
14	XTALOSC OFF	0 1	晶体振荡器关闭位：如果晶体振荡器不使用，可以使用该位将其关闭。 晶体振荡器开启（复位时的默认状态）。 晶体振荡器关闭。
13	XCLKINOF F	0 1	XCLKIN关闭位：该位关闭外部XCLKIN振荡器输入 XCLKIN振荡器输入启用（复位时的默认状态）。 XCLKIN振荡器输入关闭。 注：你需要通过XCLK寄存器的XCLKINSEL位选择XCLKIN 的GPIO引脚。更详细的信息 请见XCLK寄存器的描述。如果使用XCLKIN，XTALOSCOFF就必须置位。
12	WDHALT I	0 1	看门狗停机模式忽略位：该位选择是否通过停机模式自动开启/关闭看门狗。这个特性可以在 停机模式有效时用来允许所选的WDCLK源继续计时看门狗。这可以允许看门狗周期性地唤 醒器件。 看门狗自动通过停机模式开启/关闭（复位时的默认状态）。 看门狗忽略停机模式。
11	INTOSC2 HALTI	0 1	内部振荡器2 停机模式忽略位：该位选择是否通过停机模式自动开启/关闭内部振荡器2。这 个特性可以在停机模式有效时用来允许内部振荡器继续计时。这将使能器件更快地从停机模 式中唤醒。 内部振荡器2自动通过停机模式开启/关闭（复位时的默认状态）。 内部振荡器2忽略停机模式。
10	INTOSC2 OFF	0 1	内部振荡器2关闭位：该位关闭振荡器2 内部振荡器2开启（复位时的默认状态）。 内部振荡器2关闭。如果内部振荡器2不被使用，用户可以使用该位将其关闭。这个选择不受 缺少时钟检测电路影响。
9	INTOSC1 HALTI	0 1	内部振荡器1 停机模式忽略位：该位选择是否通过停机模式自动开启/关闭内部振荡器1。 内部振荡器1自动通过停机模式开启/关闭（复位时的默认状态）。 内部振荡器1忽略停机模式。这个特性可以在停机模式有效时用来允许内部振荡器继续计时。 这将使能器件更快地从停机模式中唤醒。

位	域	值	描述
8	INTOSC1OFF	0 1	内部振荡器1关闭位：该位关闭振荡器1。 内部振荡器1开启（复位时的默认状态）。 内部振荡器1关闭。如果内部振荡器1不被使用，用户可以使用该位将其关闭。这个选择不受缺少时钟检测电路影响。
7-5	TMR2CLKPRESCALE	000 001 010 011 100 101 110 111	CPU Timer2时钟预分频值：这些位为所选的CPU Timer2时钟源选择预分频值。这个选择不受缺少时钟检测电路影响。 /1（复位时的默认状态） /2 /4 /8 /16 保留 保留 保留
4-3	TMR2CLKSRCSEL	00 01 10 11	CPU Timer2时钟源选择位：该位为CPU Timer2选择时钟源选择SYSCLKOUT（复位时的默认状态，绕过预分频器）。 选择外部振荡器（XOR输出）。 选择内部振荡器1。 选择内部振荡器2。这个选择不受缺少时钟检测电路影响。
2	WDCLKSRCSEL	0 1	看门狗时钟源选择位：该位选择看门狗的时钟源。在 为低时和 变为高之后，默认选择内部振荡器1。用户需要在初始化过程中选择外部振荡器或内部振荡器2。如果缺少时钟检测电路检测到缺少时钟，那么该位被强制为0，并选择内部振荡器1。用户更改该位不影响PLLCR值。 选择内部振荡器1（复位时的默认状态）。 选择外部振荡器或内部振荡器2。

位	域	值	描述
1	OSCCLKSRC2SEL	0 1	振荡器2时钟源选择位：这个位用来在内部振荡器2或外部振荡器两者之间作选择。这个选择不受缺少时钟检测电路影响。 选择外部振荡器（复位时的默认状态）。 选择内部振荡器2。
0	OSCCLKSRCSEL	0 1	振荡器时钟源选择位。这个位选择振荡器的时钟源。在 为低时和 变为高之后，默认选择内部振荡器1。用户需要在初始化过程中选择外部振荡器或内部振荡器2。只要用户使用这些位来改变时钟源，PLLCR寄存器将被自动强制为零。这阻止了潜在的PLL过冲。然后，用户必须写PLLCR寄存器来配置合适的分频比。如果必要，用户也可以使用PLLLOCKPRD寄存器来配置PLL锁定周期以缩短锁定时间。如果缺少时钟检测电路检测到缺少时钟，那么这个位被自动强制为0，并选择内部振荡器1作为时钟。PLLCR寄存器也将自动强制为0来防止所有潜在的过冲。 选择内部振荡器1（复位时的默认状态）。 选择外部振荡器或内部振荡器2。注：如果用户希望使用振荡器2或外部振荡器来计时CPU，必须应该先配置这个位，然后再写OSCCLKSRCSEL位。

PLL状态寄存器 (PLLSTS)

15		14				9		8							
NORMRDYE		保留						DIVSEL							
R-0						R/W-0									
7		6		5		4		3		2		1		0	
DIVSEL		MCLKOFF		OSCOFF		MCLKCLR		MCLKSTS		PLLOFF		保留		PLLLOCKS	
R/W-0		R/W-0		R/W-0		R/W-0		R-0		R/W-0		R-0		R-1	

图例：RW=读/写；R=只读；-n=复位后的值

PLL控制寄存器 (PLLCR)

15	4	3	0
保留			DIV
R-0			R/W-0

图例：RW=读/写；R=只读；-n=复位后的值

基于PLL的配置模式

PLL模式	注释	PLLSTS [DIVSEL] ⁽¹⁾	CLKIN和 SYSCLKOUT ⁽²⁾
PLL关闭	由用户通过置位PLLSTS寄存器的PLLOFF位来激活。在这种模式下PLL模块禁用。这对于降低系统噪声和低功率操作很有用。在进入这个模式之前，PLLCR寄存器必须先被设置成0x0000（PLL旁路）。CPU时钟（CLKIN）直接从X1/X2，X1或XCLKIN上的输入时钟获得。	0, 1 2 3	OSCCLK/4 OSCCLK/2 OSCCLK/1
PLL旁路	PLL旁路是上电时或一次外部复位（ ）后默认的PLL配置。当PLLCR寄存器被设置成0x0000，或者，在PLLCR寄存器被修改之后PLL锁定到一个新的频率时，这个模式被选择。在这个模式下，PLL本身被旁路，但PLL未关闭。	0, 1 2 3	OSCCLK/4 OSCCLK/2 OSCCLK/1
PLL启用	该模式通过将非零值n写入PLLCR寄存器来实现。当写PLLCR时，器件将切换到PLL旁路模式，直至PLL锁定。	0, 1 2	OSCCLK*n/4 OSCCLK*n/2

PLLSTS[DIVSEL]在写PLLCR之前必须为零，并且，应该只有在PLLSTS[PLLLOCKS] =1之后才能被修改

选择的输入时钟和PLLCR[DIV]位应该使PLL(VCOCLK)的输出频率是最小值50MHz。

PLL 控制寄存器 (PLLCR)

PLLCR[DIV] Value ⁽³⁾	SYSCLKOUT (CLKIN) ⁽²⁾		
	PLLSTS[DIVSEL] = 0 or 1	PLLSTS[DIVSEL] = 2	PLLSTS[DIVSEL] = 3
0000 (PLL bypass)	OSCCLK/4 (Default)	OSCCLK/2	OSCCLK/1
0001	$(OSCCLK * 1)/4$	$(OSCCLK * 1)/2$	$(OSCCLK * 1)/1$
0010	$(OSCCLK * 2)/4$	$(OSCCLK * 2)/2$	$(OSCCLK * 2)/1$
0011	$(OSCCLK * 3)/4$	$(OSCCLK * 3)/2$	$(OSCCLK * 3)/1$
0100	$(OSCCLK * 4)/4$	$(OSCCLK * 4)/2$	$(OSCCLK * 4)/1$
0101	$(OSCCLK * 5)/4$	$(OSCCLK * 5)/2$	$(OSCCLK * 5)/1$
0110	$(OSCCLK * 6)/4$	$(OSCCLK * 6)/2$	$(OSCCLK * 6)/1$
0111	$(OSCCLK * 7)/4$	$(OSCCLK * 7)/2$	$(OSCCLK * 7)/1$
1000	$(OSCCLK * 8)/4$	$(OSCCLK * 8)/2$	$(OSCCLK * 8)/1$
1001	$(OSCCLK * 9)/4$	$(OSCCLK * 9)/2$	$(OSCCLK * 9)/1$
1010	$(OSCCLK * 10)/4$	$(OSCCLK * 10)/2$	$(OSCCLK * 10)/1$
1011	$(OSCCLK * 11)/4$	$(OSCCLK * 11)/2$	$(OSCCLK * 11)/1$
1100	$(OSCCLK * 12)/4$	$(OSCCLK * 12)/2$	$(OSCCLK * 12)/1$
1101-1111	Reserved	Reserved	Reserved

Flash & OTP 相关寄存器

FOPT 确定是否开启flash流水线模式寄存器

位	域	值	描述(1)(2)(3)
15-1	保留		
0	ENPIPE	0 1	Flash 管道模式使能位。当该位被置位时 Flash 管道模式激活。管道模式通过预取指令来提高指令提取的性能。更多信息请见 14.2.2 节。 当管道模式被使能时，Flash 等待状态（分页访问和随机访问）必须大于 0。 在 Flash 器件上，ENPIPE 影响 Flash 和 OTP 内容的提取。 Flash 管道模式无效。（默认） Flash 管道模式有效。

FBANKWAIT flash 等待寄存器

位↵	域↵	值↵	描述 (1)(2)(3)↵
15-12↵	保留↵	↵	保留。↵
11-8↵	PAGEWAIT↵	↵ ↵ ↵ ↵ ↵ ↵ 0000↵ ↵ 0001↵ ↵ 0010↵ ↵ 0011↵ ↵ ...↵ 1111↵	Flash 分页读的等待状态。这些寄存器位用 CPU 时钟周期来指定 Flash 存储区一个分页读操作的等待状态数目 (0..15 个 SYSCLKOUT 周期)。更多信息请见 14.2.1 节。↵ 有关一个分页 Flash 访问所需的最短时间请见器件特定的数据手册。↵ 你必须将 RANDWAIT 的值设置成大于或等于 PAGEWAIT 的值。没有提供硬件来检测大于 RANDWAIT 的 PAGEWAIT 值。↵ 每个分页 Flash 访问使用零个等待状态，或者，每次访问共使用 1 个 SYSCLKOUT 周期。↵ 每个分页 Flash 访问使用 1 个等待状态，或者，每次访问共使用 2 个 SYSCLKOUT 周期。↵ 每个分页 Flash 访问使用 2 个等待状态，或者，每次访问共使用 3 个 SYSCLKOUT 周期。↵ 每个分页 Flash 访问使用 3 个等待状态，或者，每次访问共使用 4 个 SYSCLKOUT 周期。↵ ...↵ 每个分页 Flash 访问使用 15 个等待状态，或者，每次访问共使用 16 个 SYSCLKOUT 周期 (默认)。↵

FBANKWAIT flash 等待寄存器

7-4↵	保留↵	↵	保留↵
3-0↵	RANDWAIT↵	↵	Flash 随机读等待状态。这些寄存器位用 CPU 时钟周期来指定 Flash 存储区一个随机读操作的等待状态数目（1..15 个 SYSCLKOUT 周期）。更多信息请见 14.2.1 节。↵ 有关一个随机 Flash 访问所需的最短时间请见器件特定的数据手册。↵ RANDWAIT 的值必须设置成大于 0。也就是说，必须使用至少 1 个随机等待状态。另外，你必须将 RANDWAIT 的值设置成大于或等于 PAGEWAIT 的值。本器件将不检测和纠正大于 RANDWAIT 的 PAGEWAIT 值。↵ 0000↵ 非法值。RANDWAIT 必须设置成大于 0。↵ 0001↵ 每个随机 Flash 访问使用 1 个等待状态，或者，每次访问共使用 2 个 SYSCLKOUT 周期。↵ 0010↵ 每个随机 Flash 访问使用 2 个等待状态，或者，每次访问共使用 3 个 SYSCLKOUT 周期。↵ 0011↵ 每个随机 Flash 访问使用 3 个等待状态，或者，每次访问共使用 4 个 SYSCLKOUT 周期。↵ ...↵ ...↵ 1111↵ 每个随机 Flash 访问使用 15 个等待状态，或者，每次访问共使用 16 个 SYSCLKOUT 周期（默认）。↵

应用程序举例：

GPIO初始化：

- 1) 上拉设置
- 2) MUX设置
- 3) 方向设置
- 4) 初始值设置
- 5) 特殊引脚的设置

● 多路复用 MUX

GPIO MUX

GPAMUX1 Register Bits	Default at Reset	Peripheral Selection 1	Peripheral Selection 2	Peripheral Selection 3
	Primary I/O Function (GPAMUX1 bits = 00)	(GPAMUX1 bits = 01)	(GPAMUX1 bits = 10)	(GPAMUX1 bits = 11)
1-0	GPIO0	EPWM1A (O)	Reserved ⁽¹⁾	Reserved ⁽¹⁾
3-2	GPIO1	EPWM1B (O)	Reserved	COMP1OUT (O)
5-4	GPIO2	EPWM2A (O)	Reserved	Reserved ⁽¹⁾
7-6	GPIO3	EPWM2B (O)	Reserved	COMP2OUT (O)
9-8	GPIO4	EPWM3A (O)	Reserved	Reserved ⁽¹⁾
11-10	GPIO5	EPWM3B (O)	Reserved	ECAP1 (I/O)
13-12	GPIO6	EPWM4A (O)	EPWMSYNCl (I)	EPWMSYNCO (O)
15-14	GPIO7	EPWM4B (O)	SCIRXDA (I)	Reserved
17-16	Reserved	Reserved	Reserved	Reserved
19-18	Reserved	Reserved	Reserved	Reserved
21-20	Reserved	Reserved	Reserved	Reserved
23-22	Reserved	Reserved	Reserved	Reserved
25-24	GPIO12	TZ1 (I)	SCITXDA (O)	Reserved
27-26	Reserved	Reserved	Reserved	Reserved
29-28	Reserved	Reserved	Reserved	Reserved
31-30	Reserved	Reserved	Reserved	Reserved

GPIO MUX

GPAMUX2 Register Bits	(GPAMUX2 bits = 00)	(GPAMUX2 bits = 01)	(GPAMUX2 bits = 10)	(GPAMUX2 bits = 11)
1-0	GPIO16	SPISIMOA (I/O)	Reserved	$\overline{TZ2}$ (I)
3-2	GPIO17	SPISOMIA (I/O)	Reserved	$\overline{TZ3}$ (I)
5-4	GPIO18	SPICLKA (I/O)	SCITXDA (O)	XCLKOUT (O)
7-6	GPIO19/XCLKIN	$\overline{SPISTE A}$ (I/O)	SCIRXDA (I)	ECAP1 (I/O)
9-8	Reserved	Reserved	Reserved	Reserved
11-10	Reserved	Reserved	Reserved	Reserved
13-12	Reserved	Reserved	Reserved	Reserved
15-14	Reserved	Reserved	Reserved	Reserved
17-16	Reserved	Reserved	Reserved	Reserved
19-18	Reserved	Reserved	Reserved	Reserved
21-20	Reserved	Reserved	Reserved	Reserved
23-22	Reserved	Reserved	Reserved	Reserved
25-24	GPIO28	SCIRXDA (I)	SDAA (I/OC)	$\overline{TZ2}$ (O)
27-26	GPIO29	SCITXDA (O)	SCLA (I/OC)	$\overline{TZ3}$ (O)
29-28	Reserved	Reserved	Reserved	Reserved
31-30	Reserved	Reserved	Reserved	Reserved

GPIO MUX

	Default at Reset Primary I/O Function	Peripheral Selection 1	Peripheral Selection 2	Peripheral Selection 3
GPBMUX1 Register Bits	(GPBMUX1 bits = 00)	(GPBMUX1 bits = 01)	(GPBMUX1 bits = 10)	(GPBMUX1 bits = 11)
1,0	GPIO32	SDAA (I/OC)	EPWMSYNCl (I)	$\overline{\text{ADCSOCAO}}$ (O)
3,2	GPIO33	SCLA (I/OC)	EPWMSYNCO (O)	$\overline{\text{ADCSOCBO}}$ (O)
5,4	GPIO34	COMP2OUT (O)	Reserved	Reserved
7,6	GPIO35 (TDI)	Reserved	Reserved	Reserved
9,8	GPIO36 (TMS)	Reserved	Reserved	Reserved
11,10	GPIO37 (TDO)	Reserved	Reserved	Reserved
13,12	GPIO38/XCLKIN (TCK)	Reserved	Reserved	Reserved
15,14	Reserved	Reserved	Reserved	Reserved
17,16	Reserved	Reserved	Reserved	Reserved
19,18	Reserved	Reserved	Reserved	Reserved
21,20	Reserved	Reserved	Reserved	Reserved
23,22	Reserved	Reserved	Reserved	Reserved
25,24	Reserved	Reserved	Reserved	Reserved
27,26	Reserved	Reserved	Reserved	Reserved
29,28	Reserved	Reserved	Reserved	Reserved
31,30	Reserved	Reserved	Reserved	Reserved

- 数字 I/O 控制

GPIO 的配置步骤:

Step 1. Plan the device pin-out

-- 安排器件引脚用途

Step 2. Enable or disable internal pull-up resistor

-- 使能或禁能内部上拉电阻

Step 3. Select input qualification

-- 选择输入时钟同步

Step 4. Select the pin function

-- 选择引脚功能

Step 5. For digital general purpose I/O , select the direction of pin

-- 为数字通用I/O选择引脚的方向 输入或者输出

Step 6. Select low power mode wake-up sources

-- 选择低功率模式唤醒源(是否作为唤醒源)

Step 7. Select external interrupt sources

-- 选择外部中断源

GPIO 相关寄存器 (GPIOXINTnSEL)

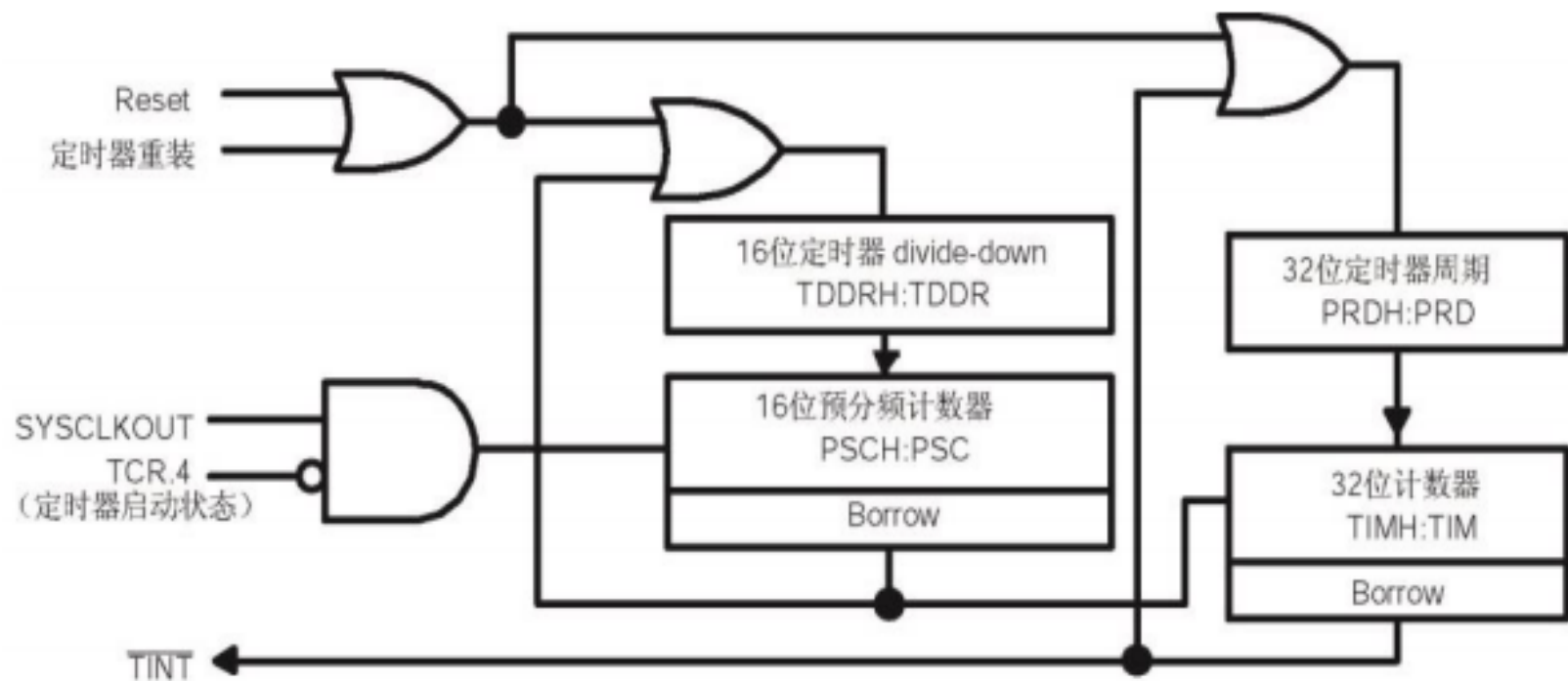
Bits	Field	Value	Description ⁽²⁾
15-5	Reserved		Reserved
4-0	GPIOXINTnSEL	Select the port A GPIO signal (GPIO0 - GPIO31) that will be used as the XINT1, XINT2, or XINT3 interrupt source. In addition, you can configure the interrupt in the XINT1CR, XINT2CR, or XINT3CR registers described in Section 6.6. To use XINT2 as ADC start of conversion, enable it in the desired ADCSOCxCTL register. The ADCSOC signal is always rising edge sensitive. 00000 Select the GPIO0 pin as the XINTn interrupt source (default) 00001 Select the GPIO1 pin as the XINTn interrupt source 11110 Select the GPIO30 pin as the XINTn interrupt source 11111 Select the GPIO31 pin as the XINTn interrupt source	

n	Interrupt	Interrupt Select Register	Configuration Register
1	XINT1	GPIOXINT1SEL	XINT1CR
2	XINT2	GPIOXINT2SEL	XINT2CR
3	XINT3	GPIOXINT3SEL	XINT3CR

应用程序举例：

CPUTimer0初始化：

- 1) 关时钟
- 2) 周期设置
- 3) 预分频设置
- 4) 初始值设置
- 5) 中断使能
- 6) 使能定时器 或 ?



定时器原理

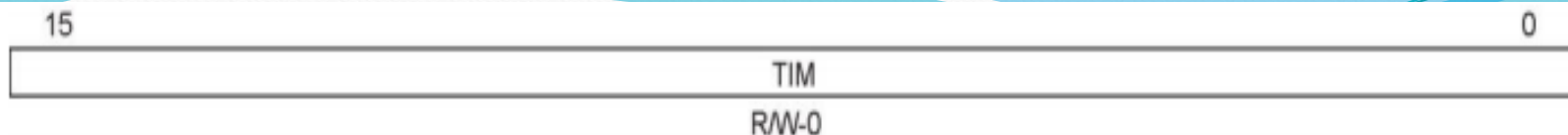
CPU 定时器的正常操作如下：32 位计数器寄存器 TIMH:TIM 装入周期寄存器 PRDH:PRD 的值。计数器寄存器的值以 28x 器件的 SYSCLKOUT 速率递减。当计数器的值到达 0 时，定时器中断输出信号产生一个中断脉冲。表 2.27 列出的寄存器用来配置定时器。

32位 CPU 计数器相关寄存器

名称	地址	大小(×16)	描述	位描述
TIMER0TIM	0x0C00	1	CPU 定时器 0, 计数器定时器	图 2.25
TIMER0TIMH	0x0C01	1	CPU 定时器 0, 计数器定时器高 16 位字	图 2.26
TIMER0PRD	0x0C02	1	CPU 定时器 0, 周期寄存器	图 2.27
TIMER0PRDH	0x0C03	1	CPU 定时器 0, 周期寄存器高 16 位字	图 2.28
TIMER0TCR	0x0C04	1	CPU 定时器 0, 控制寄存器	图 2.29
TIMER0TPR	0x0C06	1	CPU 定时器 0, 预分频寄存器	图 2.30
TIMER0TPRH	0x0C07	1	CPU 定时器 0, 预分频寄存器高 16 位字	图 2.31

续上表

名称	地址	大小(×16)	描述	位描述
TIMER1TIM	0x0C08	1	CPU 定时器 1，计数器定时器	图 2.25
TIMER1TIMH	0x0C09	1	CPU 定时器 1，计数器定时器高 16 位字	图 2.26
TIMER1PRD	0x0C0A	1	CPU 定时器 1，周期寄存器	图 2.27
TIMER1PRDH	0x0C0B	1	CPU 定时器 1，周期寄存器高 16 位字	图 2.28
TIMER1TCR	0x0C0C	1	CPU 定时器 1，控制寄存器	图 2.29
TIMER1TPR	0x0C0E	1	CPU 定时器 1，预分频寄存器	图 2.30
TIMER1TPRH	0x0C0F	1	CPU 定时器 1，预分频寄存器高 16 位字	图 2.31
TIMER2TIM	0x0C10	1	CPU 定时器 2，计数器定时器	图 2.25
TIMER2TIMH	0x0C11	1	CPU 定时器 2，计数器定时器高 16 位字	图 2.26
TIMER2PRD	0x0C12	1	CPU 定时器 2，周期寄存器	图 2.27
TIMER2PRDH	0x0C13	1	CPU 定时器 2，周期寄存器高 16 位字	图 2.28
TIMER2TCR	0x0C14	1	CPU 定时器 2，控制寄存器	图 2.29
TIMER2TPR	0x0C16	1	CPU 定时器 2，预分频寄存器	图 2.30
TIMER2TPRH	0x0C17	1	CPU 定时器 2，预分频寄存器高 16 位字	图 2.31



图例：R/W=读/写；R=只读；-n=复位后的值

图 2.25 TIMERxTIM 寄存器 (x=1, 2, 3)

位	域	描述
15-0	TIM	CPU 定时器计数器寄存器 (TIMH:TIM): TIM 寄存器保存定时器当前 32 位计数值的低 16 位。TIMH 保存定时器当前 32 位计数值的高 16 位。TIMH: TIM 每 (TDDRH:TDDR+1) 个时钟周期递减一次, 这里的 TDDRH:TDDR 是预分频 divide-down 值。当 TIMH:TIM 递减到零时, TIMH:TIM 寄存器装入 PRDH:PRD 寄存器包含的周期值。产生定时器中断 (TINT) 信号。

32位 CPU 计数器寄存器 (TIMERxTIM)

TIMERxTIMH与之类似



图例：R/W=读/写；R=只读；-n=复位后的值

图 2.27 TIMERxPRD 寄存器 (x=1, 2, 3)

表 2.30 TIMERxPRD 寄存器域描述

位	域	描述
15-0	PRD	CPU 定时器周期寄存器 (PRDH:PRD)：PRD 寄存器保存 32 位周期值的低 16 位。PRDH 寄存器保存 32 位周期值的高 16 位。当 TIMH:TIM 递减到零时，在下个定时器输入时钟周期（预分频器的输出）开始时 TIMH:TIM 寄存器装入 PRDH:PRD 寄存器包含的周期值。当定时器控制寄存器 (TCR) 的定时器重装位 (TRB) 置位时，PRDH:PRD 的内容也被装入到 TIMH:TIM 中。

32位 CPU 周期寄存器 (TIMERxPRD)

TIMERxPRDH与之类似



图例：R/W=读/写；R=只读；-n=复位后的值

图 2.29 TIMERxTCR 寄存器 (x=1, 2, 3)

表 2.32 TIMERxTCR 寄存器域描述

位	域	值	描述
15	TIF	0 1	CPU 定时器中断标志。 CPU 定时器还未递减到零。 写入 0 被忽略。 CPU 定时器递减到零时该标志置位。 通过写入 1 到这个位来清除相应的标志。
14	TIE	0 1	CPU 定时器中断使能。 CPU 定时器中断被禁能。 CPU 定时器中断被使能。如果定时器递减到零并且 TIE 置位，定时器就发出中断请求。
13-12	保留		保留

位	域	值	描述
11-10	FREE SOFT		CPU 定时器仿真模式：这两位是特殊的仿真位，当在高级语言调试器中碰到断点时，使用它们来确定定时器的状态。如果 FREE 位被置位，那么，在出现软件断点时，定时器继续运行（即，自由运行）。在这种情况下，SOFT 的值无关紧要。但是如果 FREE 为 0，SOFT 就生效。这时，如果 SOFT=0，定时器在下次 TIMH:TIM 递减时终止。如果 SOFT=1，定时器在 TIMH:TIM 已经递减到零时终止。
		FREE SOFT	CPU 定时器仿真模式
		0 0	在 TIMH:TIM 下次递减之后停止（硬停止）
		0 1	在 TIMH:TIM 递减到零之后停止（软停止）
		1 0	自由运行
		1 1	自由运行
			在软停止模式，定时器在关断之前产生一个中断（因为计数值到达 0 是产生中断的条件）。
9-6	保留		保留

TIMERxTCR寄存器描述_续

9-6	保留		保留
5	TRB	0 1	CPU 定时器重装位。 TRB 位读出时总为零。写 0 被忽略。 当写 1 到 TRB 时，TIMH:TIM 装入 PRDH:PRD 的值，并且，预分频器计数器 (PSCH:PSC) 装入定时器 divide-down 寄存器的值 (TDDRH:TDDR)。
4	TSS	0 1	CPU 定时器停止状态位。TSS 是一个 1 位的标志，用来停止或启动 CPU 定时器。 读出 0 表示 CPU 定时器正在运行。 将 TSS 设置为 0 来启动或重启 CPU 定时器。复位时，TSS 被清零，CPU 定时器立刻启动。 读出 1 表示 CPU 定时器被停止。 将 TSS 设置为 1 来停止 CPU 定时器。
3-0	保留		保留

TIMERxTCR寄存器描述_续

PSC								TDDR							
R-0								R/W-0							

图例：R/W=读/写；R=只读；-n=复位后的值

图 2.30 TIMERxTPR 寄存器 (x=1, 2, 3)

表 2.33 TIMERxTPR 寄存器域描述

位	域	描述
15-8	PSC	CPU 定时器预分频计数器。这些保存定时器的当前预分频计数值。对于 PSCH:PSC 值大于 0 的每个定时器时钟源周期，PSCH:PSC 都减 1。在 PSCH:PSC 到达 0 之后的一个定时器时钟（定时器预分频器的输出）周期，PSCH:PSC 装入 TDDRH:TDDR 的内容，并且，定时器计数器寄存器（TIMH:TIM）减 1。只要软件置位定时器重装位（TRB），PSCH:PSC 也被重装。PSCH:PSC 可以通过读寄存器来检查，但它不能直接被设置。它必须从定时器 divide-down 寄存器（TDDRH:TDDR）获取值。复位时，PSCH:PSC 被设为 0。
7-0	TDDR	CPU 定时器 Divide-Down。每隔（TDDRH:TDDR+1）个定时器时钟源周期，定时器计数器寄存器（TIMH:TIM）就减 1。复位时，TDDRH:TDDR 位被清零。为了使整个定时器计数值增加一个整数因子，将该整数因子减 1 后写入 TDDRH:TDDR 位。当预分频器计数器（PSCH:PSC）值为 0 时，一个定时器时钟源周期之后，TDDRH:TDDR 的内容重新装入 PSCH:PSC，并且 TIMH:TIM 减 1。只要软件将定时器重装位（TRB）置位，TDDRH:TDDR 也重装 PSCH:PSC。

TIMERxTPR寄存器描述 （TIMERxTPRH寄存器与之类似）

应用程序举例：

Xint1初始化：

- 1) Xint1引脚选择**
- 2) 外部中断极性选择**
- 3) 中断使能**

外部中断控制寄存器:

1. 支持3个外部中断，**XINT1 - XINT3**。
2. 每个外部中断可以选择负边沿或正边沿触发，也可以被使能或禁能。
3. 屏蔽的中断还包含一个16位自由运行的递增计数器，在检测到一个有效的中断沿时复位为0。这个计数器可以用来精确地计时中断。

中断控制和计数器寄存器（不受EALLOW保护）

名称	地址范围	大小 (x16)	描述
XINT1CR	0x0000 7070	1	XINT1 配置寄存器
XINT2CR	0x0000 7071	1	XINT2 配置寄存器
XINT3CR	0x0000 7072	1	XINT3 配置寄存器
保留	0x0000 7073 - 0x0000 7077	5	
XINT1CTR	0x0000 7078	1	XINT1 计数器寄存器
XINT2CTR	0x0000 7079	1	XINT2 计数器寄存器
XINT3CTR	0x0000 707A	1	XINT3 计数器寄存器
保留	0x0000 707B - 0x0000 707E	5	

15	4	3	2	1	0
保留	Polarity		保留	Enable	
R-0	RW-0		R-0	R/W-0	

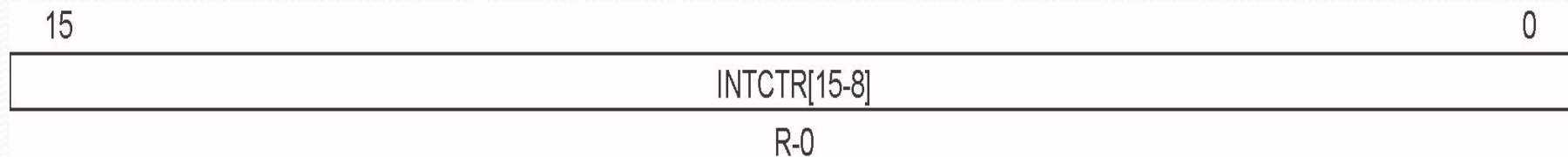
图例：RW=读/写；R=只读；-n=复位后的值

图 3.13 外部中断n控制寄存器（XINTnCR）

表 3.15 外部中断n控制寄存器（XINTnCR）的域描述

位	域	值	描述
15-4	保留		读这些位返回 0；写这些位没有任何影响。
3-2	Polarity	00	中断在下降沿（高到低的跳变）产生。
		01	中断在上升沿（低到高的跳变）产生。
		10	中断在下降沿（高到低的跳变）产生。
		11	中断在下降沿以及上升沿（高到低的跳变以及低到高的跳变）产生。
1	保留		读该位返回 0；写该位没有任何影响。
0	Enable	0	这个读/写位使能或禁能外部中断 XINTn。 禁能中断。
		1	使能中断。

对于XINT1/XINT2/XINT3来说，还有一个16位的计数器，只要检测到一个中断边沿，计数器就复位为0x000。这些计数器可以用来精确计时中断的出现。



图例：RW=读/写；R=只读；-n=复位后的值

外部中断n计数器（XINTnCTR）的域描述

位	域	描述
15-0	INTCTR	这是一个自由运行的 16 位递增计数器，它以 SYSCLKOUT 的速率被计时。计数器的值在检测到一个有效的中断边沿时复位到 0x0000，然后继续计数，直至检测到下个有效的中断边沿。当中断被禁能时，计数器停止运行。计数器是一个自由运行的计数器，达到最大值时返回到 0。计数器是一个只读寄存器，只有一个有效的中断边沿或复位能将其复位到零。

中断初始化

// Step 3. Clear all interrupts and initialize PIE vector table: Disable CPU interrupts

```
DINT;  
InitPieCtrl();  
IER = 0x0000;  
InitPieVectTable();
```

用到的中断向量填充

```
ENPIE = 1  
PIEIERx的设置  
IER的设置  
EINT;
```


总体要求：做一个计时器，分辨率到0.01秒

具体：1) 显示XX.XX（小时与分钟）

XX.XX（秒与0.01秒）

2) 按键启停：启动—>停止—>清零—>启动

3) 跑马灯

变量定义:

```
int hourH = 0, hourL = 0;
```

```
int minH = 0, minL = 0;
```

```
int secH = 0, secL = 0, TenmS = 0;
```

```
int Running = 0; // 0=等待 1=计时 2=停止
```

```
int NewLedEn = 0; // 0=显示更新允许 1=已经更新
```

```
int KeyDLTime = 0; // 按键去抖动用
```

```
int LedFlashCtr; // 用显示更新和跑马灯控制
```

跑马灯的宏定义:

```
#define Led0Blink() GpioDataRegs.GPACLEAR.bit.GPIO0 = 1
```

```
#define Led1Blink() GpioDataRegs.GPACLEAR.bit.GPIO1 = 1
```

```
#define Led2Blink() GpioDataRegs.GPACLEAR.bit.GPIO2 = 1
```

```
#define Led3Blink() GpioDataRegs.GPACLEAR.bit.GPIO3 = 1
```

```
#define Led0Blank() GpioDataRegs.GPASET.bit.GPIO0 = 1
```

```
#define Led1Blank() GpioDataRegs.GPASET.bit.GPIO1 = 1
```

```
#define Led2Blank() GpioDataRegs.GPASET.bit.GPIO2 = 1
```

```
#define Led3Blank() GpioDataRegs.GPASET.bit.GPIO3 = 1
```

定时器初始化:

```
InitCpuTimers(); // 初始化时钟
```

```
ConfigCpuTimer(&CpuTimer0, 60, (1));
```

```
EALLOW;
```

```
CpuTimer0Regs.TCR.bit.TRB = (2); //重载
```

```
CpuTimer0Regs.TCR.bit.TSS = (3); //启动
```

```
EDIS;
```


跑马灯用IO引脚初试化：

EALLOW;

GpioDataRegs.GPASET.bit.GPIO0 = 1;

GpioDataRegs.GPASET.bit.GPIO1 = 1;

GpioDataRegs.GPASET.bit.GPIO2 = 1;

GpioDataRegs.GPASET.bit.GPIO3 = 1;

GpioCtrlRegs.GPAMUX1.bit.GPIO0 = (4);

GpioCtrlRegs.GPAMUX1.bit.GPIO1 = (4);

GpioCtrlRegs.GPAMUX1.bit.GPIO2 = (4);

GpioCtrlRegs.GPAMUX1.bit.GPIO3 = (4);

GpioCtrlRegs.GPADIR.bit.GPIO0 = (5);

GpioCtrlRegs.GPADIR.bit.GPIO1 = (5);

GpioCtrlRegs.GPADIR.bit.GPIO2 = (5);

GpioCtrlRegs.GPADIR.bit.GPIO3 = (5);

EDIS;


```
void Xint1_Init()
{

    EALLOW;
    GpioCtrlRegs.GPAMUX1.bit.GPIO12 = __ (6) __;
    GpioCtrlRegs.GPADIR.bit.GPIO12 = __ (7) __;
    GpioCtrlRegs.GPAPUD.bit.GPIO12 = __ (8) __; // Pullup's enabled GPIO12
    GpioIntRegs.GPIOXINT1SEL.bit.GPIOSEL = __ (9) __;
    XIntruptRegs.XINT1CR.bit.POLARITY = __ (10) __; // 下降沿
    XIntruptRegs.XINT1CR.bit.ENABLE = __ (11) __; // 允许
    EDIS;
}
```

应用程序举例：

Xint1中断服务程序：

如果目前是等待状态，重置定时器，启动

如果目前是计时状态，则计时禁止（停止）

如果目前是计时停止状态，则清零计时值，进入等待状态

```
PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
```



```
interrupt void myXint1_isr(void){
    if((Running == 0)&&(KeyDLTime > 20)){
        EALLOW;
        CpuTimer0Regs.TCR.bit.TSS = 1;
        CpuTimer0Regs.TCR.bit.TRB = 1;
        CpuTimer0Regs.TCR.bit.TSS = 0;
        EDIS; Running = 1; KeyDLTime = 0;
    }
    else if((Running == 1)&&(KeyDLTime > 20)){
        Running = 2;
        KeyDLTime = 0;
    }
    else if((Running == 2)&&(KeyDLTime > 20)) {
        Running = 0; hourH=0;hourL=0;minH=0;minL=0;
        secH=0;secL=0;TenmS = 0; StopTime = 0;
        KeyDLTime = 0;
    }
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
}
```


应用程序举例:

CPUTimer0中断服务程序:

显示更新允许

XINT1间隔相关

如果计时允许{

0.01S计数器加1

如果: 0.01S计数器=100, 清零, 1S计数器加1

如果: 1S计数器=60, 清零, 1M计数器加1

如果: 1M计数器=60, 清零, 1H计数器加1

}

跑马灯程序

PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;

```
interrupt void cpu_timer0_isr(void) {
    KeyDLTime++;
    LedFlashCtr++;
    if((LedFlashCtr & 0xf)==0)NewLedEn = 0;
    if(Running == 1){
        TenmS++;
        if(TenmS == 100){ TenmS = 0;      secL++; }
        if(secL==10){ secL=0; secH++; }
        if(secH==6){ secH=0; minL++; }
        if(minL==10){ minL=0; minH++; }
        if(minH==6){ minH=0; hourL++; }
        if(hourL==4 && hourH==2){ hourL=0; hourH=0; }
        else if(hourL==10){ hourL=0; hourH++; }
    }
    HorseRunning((LedFlashCtr & 0xf0)>>4);
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
}
```


应用程序举例：

主程序： **main** ()

{

系统、外设、中断初始化；

计时变量初始化；

显示初始化

while (1) {

 如果显示更新允许{

 清除更新允许；

 显示；

 }

}

}

应用程序举例:

连接命令文件: F28027.cmd

特别注意: L0的运用问题

Main():

```
// MemCopy(&RamfuncsLoadStart, &RamfuncsLoadEnd,  
&RamfuncsRunStart);
```

F28027.cmd:

```
codestart      : > BEGIN      PAGE = 0  
/*  
  ramfuncs      : LOAD = FLASHA,  
                  RUN = PRAML0,  
                  LOAD_START(_RamfuncsLoadStart),  
                  LOAD_END(_RamfuncsLoadEnd),  
                  RUN_START(_RamfuncsRunStart),  
                  PAGE = 0  
*/
```

P.151作业第三题

```
void ByteOutput(unsigned char out)
{
    int i,flag;
    flag =0x1 ;
    for(i=0;i<8;i++){
        if((out & flag) != 0) GpioDataRegs.GPASET.all |= flag;
        else GpioDataRegs.GPACLEAR.all |= flag;
        flag = flag + flag;
    }
}
```




P.151作业第三题

```
void ByteOutput(unsigned char out)
{
```

```
GpioDataRegs.GPASET.all = (unsigned long int)(out & 0xff);
GpioDataRegs.GPACLEAR.all = (unsigned long int) ((~out) & 0xff) ;
}
```